

Lecture 23

C Programming III

CS 12 Team 25.2
A.Y. 2025–2026, 2nd Sem

Department of Computer Science
College of Engineering
University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Structs

structS

Struct: Composite data type representing a group of values with fixed names (with possibly different types)

- C version of Python `objects` (no methods)

structS

```
// Declaration of new kind of struct type
struct person {
    char name[50];
    int year;
};

int main() {
    // Uninitialized struct (contains garbage)
    struct person x;

    x.name = "Rosa";
}
```

```
x.year = 1949;
```

```
printf("Name: %s\n", x.name);
```

```
// Initialized struct
```

```
struct person y = {.name = "Rudolf",  
                   .year = 1941};
```

```
}
```

structS

```
// `Person` as alias for `struct person`;  
// `struct person` can still be used if needed  
typedef struct person Person;  
struct person {  
    char name[50];  
    int year;  
};  
int main() {  
    // typedef alias is conventionally capitalized  
    Person x = {.name = "Ange", .year = 1980};  
}
```

structS

```
// Version with no separate statement;  
// no `struct person` defined (not ideal for  
// recursively defined structs)  
typedef struct {  
    char name[50];  
    int year;  
} Person;
```


Aside: Parameters and Return Values

```
#include <stdio.h>
```

```
int f(int x) {  
    x += 1;  
  
    return x;  
}
```

```
int main() {  
    int x = 10;  
    int y = 20;
```

```
printf("%d\n", x); // 10
printf("%d\n", y); // 20
y = f(x);
printf("%d\n", x); // 10
printf("%d\n", y); // 11
```

```
}
```

Aside: Parameters and Return values

- Parameters are local variables
- Arguments are copied as is to parameters
- Return value is copied as is to caller variable
- Local variables are deallocated on function return

Passing and returning structs

```
typedef struct {  
    char name[10];  
    int year;  
} Person;
```

```
void f_wrong(Person z) {  
    z.year += 1;  
}
```

```
Person f_slow(Person z) {  
    z.year += 1;
```

```
    return z;
}

int main() {
    Person x = {.name = "Krauss", .year = 1936};

    f_wrong(x);    // `x` is unaffected; 1x copy
    x = f_slow(x); // `x` is overwritten; 2x copy
}
```

Passing and returning `structs`

Visualization: [Link](#)

- Structs are copied as is to parameters
 - Slow for larger structs
- Returned `struct` is copied as is to caller variable
 - Slow for larger structs

How do we update `structs` of one function in another without copying the entire `struct`?

Copy-free `struct` updates via pointers

Pointers enable passing and updating of `struct` values without copying the entire `struct`

Copy-free struct updates via pointers

```
typedef struct {  
    char name[50];  
    int year;  
} Person;  
  
void f1(int *year) {  
    *year = *year + 1;  
}  
  
void f2(Person *p) {  
    int *q = &(*p).year; // &((*p).year)
```



```
    *q = (*p).year + 1;
}

void f3(Person *p) {
    int *q = &p->year; // &(p->year)
    *q = p->year + 1;
}
```

Copy-free `struct` updates via pointers

```
int main() {  
    Person x = {.name = "Krauss", .year = 1936};  
  
    f1(&x.year);  
    f2(&x);  
    f3(&x);  
}
```

Dynamic memory allocation

Motivating problem

Recall that returning a pointer to a local variable should not be done:

```
#include <stdio.h>
#include <stdint.h>

int *f() {
    int x = 12;

    return &x;
}
```

```
int main() {  
    f();  
}
```

How do we make the called function return data to its caller without using an output parameter?

`malloc`

`malloc`: Built-in function in `stdlib.h` that enables allocating of heap memory

- `malloc(<size to allocate>)`
 - Returns a pointer to the allocated memory (first address)
 - NULL on failure
 - Receiving variable is pointer to any type
 - Data is uninitialized
 - Alternative: `calloc(<n>, <size_each>)` (init all with "zero")

- `sizeof(<type>)` can be used to get size of type

malloc

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Alternative to local variable allocation;
    // `p` is a local variable, but points to
    // nonlocal memory
    int *p = malloc(sizeof(int));
    int *q = malloc(sizeof(int));
    scanf("%d", p);
    *q = 12;
```



```
printf("Sum: %d\n", *p + *q);  
}
```

malloc

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Alternative to local array allocation;
    // nonlocal memory with space for 10 `int`s
    int *nums = malloc(sizeof(int) * 10);

    for (int i = 0; i < 10; i++) {
        scanf("%d", &nums[i]);
    }
}
```

Aside: Stack vs Heap memory

Stack memory: region of memory used for call stack and local variables

- “Faster” to access (CS 21/155)
- Smaller default limit (~8 MB on Linux)
- In C:
 - Allocations active until end of function
 - Deallocated automatically on function end

Aside: Stack vs Heap memory

Heap memory: region of memory used for dynamic allocation

- “Slower” to access (CS 21/155)
- Larger default limit (“unlimited”; CS 140)
- In C:
 - Manual allocation via `malloc`
 - Allocations active until end of program
 - Manual deallocation via `free`

More details in CS 21/140/155

free

`free(<ptr>)`: Deallocates heap-allocated memory pointed to by `<ptr>`

- Should `free` heap allocation when done with it (next slide answers why)
- Garbage collection: Automatic calling of `free` (manually done in C)
 - Python/JS runtime manages `malloc/free` calls for programmer convenience

`free`

Warning: Do not do the following (undefined behavior; likely crash)

Use-after-free (UAF): Accessing memory deallocated by `free`

```
int *p = malloc(sizeof(int));  
free(p);  
*p = 12; // Dangling pointer; do not dereference
```

`free`

Double free: Calling `free` with free'd pointer

```
int *p = malloc(sizeof(int));  
free(p);  
free(p);
```

Hard to manage for more complex programs; cause of serious browser security bugs: [CVE-2025-3066](#), [CVE-2023-42950](#) etc.

Memory leaks

Memory leak: Heap-allocated memory that cannot be manually deallocated due to losing access to all references to it

```
#include <stdio.h>
#include <stdlib.h>

void f(int n) {
    int *nums = malloc(sizeof(int) * n);

    for (int i = 0; i < n; i++) {
        nums[i] = 12;
    }
}
```



```
}
```

```
printf("Done for %d integers\n", n);
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    while (scanf("%d", &n) == 1) {
```

```
        if (n <= 0) break;
```

```
        f(n);
```

```
    }
```

```
}
```

Memory leaks

Heap-allocated memory is not automatically deallocated while program is running

- Deallocated when program terminates
- Every call to `f` allocates:
 - Stack: `n`, `nums`, `i`
 - Heap: Block of size `n`

Memory leaks

- When call to `f` completes:
 - `n`, `nums`, `i` are deallocated
 - Heap block of size `n` remains allocated

Always remember to free all `malloc` allocations!